

M. Moral Corp.

A Cognitive Bias Training Game

Youtube link: https://youtu.be/dTENH_fkxdc

Ameya Ghuman

MSc Computer Game Engineering

Newcastle University

Newcastle Upon Tyne, Tyne and Wear, United Kingdom

c4011955@ncl.ac.uk

ABSTRACT

1 Introduction

Cognitive biases influence human judgment in subtle ways, affecting everyday decision-making. These biases are typically studied through controlled experiments, but video games provide a unique medium for uncovering and even altering biases in an interactive manner. The purpose of this project is to investigate if it is possible to profile a player's susceptibility to various cognitive biases purely through gameplay interaction, without having to explicitly quiz the player, breaking immersion, or even have the player realize the purpose of each interaction.

The author's aim with this project was to create an immersive 3D walking simulator, stylistically like *The Stanley Parable*, in which the player can freely roam a level and come across a series of explorable areas and puzzles.

A key design challenge was in ensuring each "mini-test" was engaging to the player and was seamless within the game world, so the player remained unaware of the profiling taking place in each interaction. Voiceovers, sound design, and visual storytelling were used to create environmental feel and to subtly feed the player the required information for each test, while informing the player that they are free to interact with the world in any manner they choose.

This project also served as a personal learning exercise in Unreal Engine 5, as one of the key objectives of the author was to experiment with as many common game development techniques and features as possible. Cutscenes, animations, subtitled voiceovers, a branching dialogue system, interactive objects, and an in-world video playing system were some key features that were implemented using blueprints. This wide exploration allowed the author to gain a deeper understanding of Unreal Engine 5 and how individual systems work together to create a unique experience when combined.

1.1 Objectives

The objectives of the project were as follows:

1. **To design puzzles** that test a player's susceptibility to specific cognitive biases through diegetic game interactions.

2. **To construct an immersive experience** where psychological profiling occurs naturally, without the player feeling any pressure to make a certain decision over another.
3. **To explore diverse game development techniques** in Unreal Engine 5 using blueprints.
4. **To create a humorous but consistent atmosphere** that enhances immersion while showcasing psychological concepts in an engaging way.

CCS CONCEPTS

- Human-centered computing → Empirical studies in HCI;
- Applied computing → Interactive learning environments;
- Software and its engineering → Designing software.

KEYWORDS

Cognitive biases, puzzle design, game-based learning, Unreal Engine

ACM Reference format:

Ameya Ghuman. 2025. M Moral Corp.: A Cognitive Bias Training Game. In *Proceedings of ACM Woodstock conference (WOODSTOCK'18)*. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/1234567890>

2 Background & Related Work

2.1 Conceptual Research

This project draws inspiration from two key sources: The walking simulator *The Stanley Parable* [1], and the book on cognitive psychology *Thinking, Fast and Slow* [2] by Daniel Kahneman.

The Stanley Parable is a game in which the player has very little to "do" in the conventional sense. There is no flash and no exciting set pieces. Despite its minimalism, the game was both critically and commercially successful, largely due to its intricately designed game world, strong writing, and clever use of visual storytelling. It proved that if a game has a well-designed tone and style, it can engage audiences. The idea of doing more with less is one the author has attempted to replicate in this project.

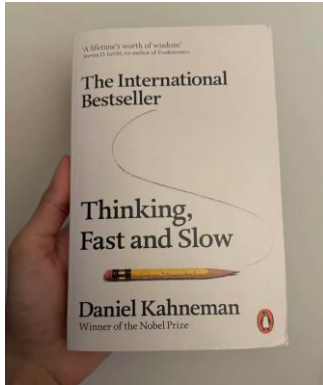


Figure 1: The author's copy of Daniel Kahneman's *Thinking, Fast and Slow*.

Thinking, Fast and Slow was the author's entry into cognitive biases and the studies of cognitive biases. It touches on a variety of systematic errors in human reasoning, which are attributed to the brain's two processing systems. One fast and one slow. The concepts presented in the book were fascinating and this project was conceptualized from the idea of taking some of those biases and building original puzzles that would test players' susceptibility to those biases in a diegetic way.

The project aims to combine the highly immersive storytelling of *The Stanley Parable* with the real-world concepts depicted in *Thinking, Fast and Slow*.

2.2 Map Design

The initial idea behind the level design was to create a *BackroomsLevel0* environment and have the player escape the backrooms by completing all mini-tests and finding the exit. The full map was constructed by following the video tutorial from the channel *UnrealityBites* [3]. Once completed, it became clear that the *Backrooms* aesthetic came with a sense of dread, an atmosphere that could affect the decision-making of the player. Since the aim of the project was to observe the player's cognitive biases through their decisions, it was important to make the environment a non-factor in the player's decision-making.

At this point, inspiration was drawn from the TV Series *Severance*, which features an office intentionally designed to strip away emotion and keep external stimuli to a minimum. The map layout from the original *Backrooms* design remained, but the materials, textures, lighting and colour palette were replaced to match the design language of *Severance*.

Further creative inspiration was drawn from *Severance* when naming certain other non-playable characters in the game.



Figure 2: The muted design of the Lumon office as depicted in the Apple TV series *Severance*. Photo: Courtesy of Apple.

2.3 Bias Research

The first challenge in designing the puzzles lay in selecting the biases to focus on from the *codex* [4]. The scope of this project dictated choosing approximately a dozen, so choosing biases that would offer the most engaging experiences when translated into game format was paramount.

Two main criteria guided the selection process. First, biases that could be communicated well through a single in-world interaction. Second, biases which would allow the construction of puzzles with new features and characteristics each time, to serve as an opportunity to deepen knowledge within the engine.

Some biases such as the *framing effect* and *anchoring bias* were considered but ultimately ruled out since implementing them meaningfully would require either multiple playthroughs, or multiple similar questions. For example, *framing effect* would require asking the same question with positive framing, and negative framing.

The resulting selection facilitated the exploration of Unreal Engine's features while staying conceptually true to the biases selected. Some puzzles are reinterpretations of real-world experiments while other puzzles are original creations based on personal understanding of the biases in question.

2.3.1 Sunk Cost Fallacy. Arkes and Blumer (1985) describe the sunk cost fallacy as "a greater tendency to continue an endeavor once an investment in money, effort, or time has been made" [5]. A widely cited experiment by Arkes and Blumer (1985) presented participants with scenarios in which they had had made prior investments of time or money and were faced with the choice to continue the activity. For example, participants were asked whether they would continue with a ski trip despite poor conditions if they had paid more for the ticket than an alternative, cheaper trip. The results demonstrated a systematic bias toward continuing the more expensive trip, illustrating that decisions can be influenced by prior investments, even when irrelevant to the best-case scenario.

2.3.2 Endowment effect. Thaler (1980) first described the endowment effect as a divide between an individual's willingness to accept (WTA) compensation for giving up an object and their willingness to pay (WTP) to acquire it [6]. A widely cited experiment by Kahneman, Knetsch, and Thaler (1990) randomly

assigned participants as either coffee mug owners or non-coffee mug owners. The owners were asked to name the minimum price at which they would sell their mug, and non-owners were asked to name the maximum price they would be willing to pay for the same mug. The results demonstrated that owners consistently valued the mugs at roughly double the price that non-owners were willing to pay [7].

2.3.3 Social Proof. Cialdini (2007) defines social proof as the principle that “we view a behavior as correct in a given situation to the degree that we see others performing it” [8]. A widely cited experiment by Milgram, Bickman, and Berkowitz (1969) instructed a few people to stand on a busy sidewalk and look up at a building. As the number of people increased, the number of passersby who also stopped and looked up increased [9].

2.3.4 Gambler's Fallacy. Tversky and Kahneman (1971) describe gambler's fallacy as the mistaken belief that the probability of a random event is influenced by prior independent outcomes [10]. A widely cited experiment by Tversky and Kahneman (1971) showed participants sequences of binary events (such as coin toss results) and asked to predict future outcomes. Even when explicitly told the events were random, many participants predicted that an overbearing amount of one outcome (e.g., several consecutive heads) made it more likely for the opposite outcome to occur next [10].

2.3.5 Reverse Psychology & Reactance. Reverse psychology is a persuasion technique which involves asserting behaviour that is opposite to the desired one, with the expectation that the motivation to adopt the desired behaviour is increased. This approach leverages the Reactance effect, a motivational state caused by an individual perceiving a loss of freedom (Brehm, 1966) [11]. A widely cited experiment by Brehm (1966) restricted access to certain toys for children. The experiment demonstrated the reactance effect by showing that after the restriction to those toys had been lifted, children were more likely to choose those toys. Further studies have validated this behaviour across multiple demographics by Brehm & Brehm (1981) [12].

2.3.6 Decoy Effect. Huber, Payne, and Puto (1982) describe the decoy effect as the phenomenon wherein a dominant option's choice share grows when an unevenly dominating choice is added [13]. A widely cited experiment by Huber, Payne, and Puto (1982) presented participants with two products that varied in two attributes (e.g., price and quality). When a third product was added that was clearly worse than one of the original two options, the choice share for that dominant option increased [13].

2.3.7 Hyperbolic discounting. Laibson (1997) describes hyperbolic discounting as a tendency to overvalue immediate rewards, leading to inconsistent preferences over time [14]. A widely cited experiment by Kirby and Maraković (1996) presented participants with a series of monetary rewards between smaller amounts available immediately and larger amounts available after a delay [15]. By altering the amounts of the rewards and the length of the delay, the study measured the rate at which participants devalued future rewards in favour of immediate rewards.

2.3.8 Optimism Bias. Sharot (2011) defines optimism bias as “the consistent tendency of people to overestimate the likelihood of

experiencing positive events and underestimating the likelihood of experiencing negative ones” [16]. A widely cited experiment by Weinstein (1980) asked participants to estimate their own likelihood of experiencing various positive and negative life events as compared to the likelihood of “an average person of your age and sex” experiencing the same. The results consistently showed that participants overestimated their own chances of experiencing positive events and underestimated their chances of experiencing negative events as compared to their peers [17].

2.3.9 Conjunction Fallacy. Tversky and Kahneman (1983) define the conjunction fallacy as “the erroneous belief that specific conditions are more probable than a single general one” [18]. In a widely cited experiment by Tversky and Kahneman (1983) called “the Linda problem”, participants were given a description of Linda, a 31-year-old lady, single, outspoken and deeply concerned with issues of discrimination. The participants were then asked which was more likely:

1. Linda is a bank teller, or
2. Linda is a bank teller and active in the feminist movement.

Even though the probability of two events occurring at the same time can never be higher than the probability of one event occurring alone, most participants chose the second option. This illustrates how a descriptive representation can override probabilistic reasoning [18].

2.3.10 Authority Bias. Milgram (1963) describes the authority bias as the tendency to obey orders from an authority figure, even when the order conflicts with one's morals [19]. A widely cited experiment by Milgram (1963) instructed participants to administer what they believed were electric shocks to another participant whenever the latter gave incorrect answers to a memory task. Despite hearing signs of distress from who was, a researcher on the experiment, participants continued to administer increasingly severe shocks because the instructions came from an authority figure in a lab coat [19].

2.3.11 Halo Effect. Thorndike (1920) first described the halo effect as “a tendency to think of the person in general as good or bad, and then to ascribe corresponding qualities to them” [20]. Thorndike's original investigation (1920) examined how military officers ranked their subordinates on various attributes such as physique, intelligence, leadership, and character. Rankings of these attributes were found to be highly correlated, suggesting that an overall positive or negative impression of an individual influenced the evaluation across multiple unrelated traits [20]. The study by Nisbett and Wilson (1977) then expanded upon Thorndike's investigation, extending the halo effect across multiple domains showing that attractiveness, perceived expertise, or other unrelated qualities could influence judgment in unrelated areas [21].

3 Design and Implementation

3.1 Wide accessibility

From the outset, wide accessibility was a primary design goal. The intention was to create an experience that would be easily comprehensible across a wide range of demographics and playable across a wide range of computer hardware. This shaped all technical and artistic decisions made throughout development. The Unreal Engine 5 **target hardware preset** was set to Desktop-Scalable to ensure compatibility with lower-end systems. *The Stanley Parable* served as inspiration for the core first-person walking simulator design to be approachable to wide audiences.

As mentioned in **section 2**, the level layout is a hybrid of the *Backrooms Level 0* structure and the design language of the *Severance* office. The result is a large open area without enclosed rooms that is segmented by walls. This design guarantees ease in navigation, and the minimalist aesthetic ensures that the map has little influence over the decisions the player makes.

The premise of the game is also made easy to understand. The player is depicted as an employee of a satirically evil corporation, and they are undergoing an employee survey in the employee survey facility.

The game sacrifices visual fidelity for runtime performance. In development, a hard cap of 4000 triangles was set for all static meshes in the level. Additionally, the decision was made to illuminate the entire world by the emissive properties of the ceiling material rather than physics simulated light sources. This material instance dynamic produces constant white light, without requiring any lighting calculations at runtime. No shadows are rendered either, reducing the GPU overhead, and making the world look like a dream-like state.

3.2 Core mechanics

This subsection outlines the mechanics that are most prominent in the game. Whether used in puzzle construction or world building, these mechanics are the most prevalent systems in the game.

3.2.1 E to interact. An ‘E to interact’ model is implemented to prioritize accessibility. Pressing the *E Key* draws a linetrace in the direction of the first-person camera view. If the linetrace hits an interactable object, the object triggers its own blueprint functionality. Interactable objects are visually indicated by a widget when the player nears one, drawn in screen space to make sure it faces the player in any direction.



Figure 3: The E to interact widget on the coin-flip puzzle.

3.2.2 Dedicated puzzle cameras. Several puzzles have dedicated camera views that the game view switches to in one of two ways. In the first way, the camera transitions to a predefined view that frames the puzzle in its entirety while a voiceover plays to deliver the player essential information in the context of that puzzle. This allows the player to understand the task at hand before the game view switches back to the first-person camera. In the second way, some puzzles transition the game view to the puzzle’s dedicated camera to offer the player an optimal angle to interact with the puzzle beyond what the first-person camera can afford.

3.2.3 OnClick, MouseOver, & Left Click Events. Mouse interaction is used primarily in the second camera-view approach as mentioned in **section 3.2.2**. When in the puzzle camera view, the mouse cursor is shown on screen and *mouseOver* events are enabled, which allows the game to detect whether a clicked component matches the intended target of interaction. This approach prevents unintended interactions in multi-component puzzles. When the game view switches back to the first-person camera, the cursor is hidden and *mouseOver* events are disabled. *OnClick* events are heavily used in menu navigation and widget interaction. In the “Killing Helly” puzzle, a boolean variable temporarily enables a linetrace to be drawn in the camera view direction upon pressing left-click to detect whether the player is aiming at Helly.

3.2.4 Collision zones. Collision zones are used widely in the game for both puzzle-specific interactions and environmental events. This approach allows interactions to occur automatically when the player collides with the zone, eliminating the need for any input. Some puzzles use these collision zones as a defined area within which the puzzle takes place. As such, there are various *OnBeginOverlap* and *OnEndOverlap* functions for each puzzle. One common function used in all puzzles’ *OnEndOverlap* is the *CheckIfDone* function. This function returns a Boolean value that determines whether the player has met all the criteria to be able to end the game. If so, a voiceover is played to let the player know.

3.2.5 Voiceovers with subtitles. Voiceovers have been used to deliver puzzle instructions and to set the overall tone of the game, inspired by the delivery style of *The Stanley Parable*, they provide a means of integrating humour with contextual information in a diegetic manner to keep the player immersed. Each dialogue in a voiceover is subtitled and time-coded for accessibility. In addition, the subtitles also serve as a stylistic choice. The combination of voiceovers with events allows the game to feel responsive to player actions.

3.2.6 Cutscenes. The opening cutscene following the generator puzzle is implemented using Unreal Engine’s built in cinematic sequences to explain the premise of the game to the player. Further cutscenes are implemented using the **first dedicated puzzle camera approach as described in section 3.2.2** where the game view temporarily switches to a dedicated camera that frames the puzzle, while delivering a voiceover that feeds important contextual information to the player. This allows the game to set tonal feel while keeping the player immersed.

3.2.7 Game instance for persistent data. A game instance class is used to store puzzle results and the *player bias profile*. This

design is necessary to be able to access the profile screen from both the *MainMenu* level and the gameplay level. It avoids the need for external save files and ensures data stays consistent between levels, reflecting changes in subsequent playthroughs.

3.2.8 Sounds. Sound design is treated as a core mechanic since it plays a huge role in atmospheric feel and tactile feedback. All sounds are sourced *freesound.org* and edited in *Audacity* to suit their use in-game. Each sound effect is deliberately triggered to play to an action or an event such that every interaction produces an appropriate auditory response.

3.3 Puzzle implementation

3.3.1 Generator puzzle. The generator puzzle was designed to measure the player's susceptibility to the sunk cost fallacy. In the study by Arkes and Blumer (1985), participants were more likely to continue with an activity if they had invested more resources into it, even when the investment was irrelevant to achieving the best-case scenario [5]. This puzzle attempts to recreate the conditions set in that study, replacing the monetary investment with an investment of time and effort.

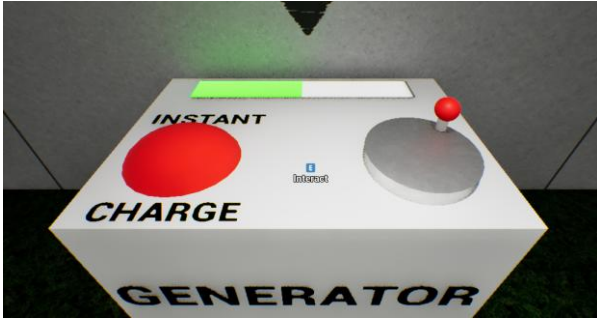


Figure 4: The generator puzzle as viewed from the generator camera.

The game begins in a dark room with the generator being the only object visible to the player, lit by a single cube in the ceiling with an emissive light. When the player approaches the generator, the *E to interact* widget appears on the screen. Upon interaction, the camera transitions to the dedicated generator camera (*GeneratorCam*), *mouseOver* events are enabled, and the cursor is shown on screen. The generator is comprised of four key elements. A mechanical crank that has a handle with a red 'grabber', a red button labelled 'Instant Charge' (initially unlit), a progress bar (initially empty), and a white cube static mesh which serves as the housing.

As mentioned in **section 3.1**, the world is illuminated by the emissive properties of the ceiling. This is achieved by multiplying an RGB value of (1,1,1) with a scalar parameter *EmissiveStrength* and hooking that up to the emissive lighting property of the ceiling's material instance dynamic. *EmissiveStrength* starts at 0.0, hence the room being dark. When the player clicks on the 'grabber', a timeline-driven *Add World Rotation* animates the crank to rotate clockwise, at the end of which the progress bar is incremented by 4% and *EmissiveStrength* is simultaneously incremented by 0.04. These two increments provide direct visual feedback to player action.

Once the generator reaches 80% charge, equivalent to 20 crank rotations, the instant charge button lights up and plays a sound indicating that it can be pressed. At this stage, the player faces a choice, continuing to rotate the crank or pressing the button. Continuing to rotate the crank at this stage would be a behavioural indicator of the sunk cost fallacy, as the player has already invested effort and may perceive pressing the button as a waste of that effort. A Boolean variable *SunkCostFallacy*, stored in the game instance, records whether the player presses the button.

3.3.2 Cup Swap puzzle. The cup swap puzzle was designed to measure the player's susceptibility to the endowment effect. In the study by Kahneman, Knetsch, and Thaler (1990), coffee mug owners valued the mugs at roughly double the price that non-owners were willing to pay [7]. This puzzle attempts to recreate the premise of the study, by giving the player a mug as a gift and asking the player whether they would swap their mug for an identical one.



Figure 5: The cup swap puzzle.

This puzzle is split into two stages. The first stage starts with the opening cutscene which plays directly after the player has finished the generator puzzle. In the cutscene, the player is shown the cup is directly behind their position and told in voiceover that it is a gift for taking part in the survey. The voiceover also mentions that it is mandatory for all employees to pick it up. The framing of the voiceover implies a gesture of kindness, yet also a sense of obligation to pick up the cup, intended to create a sense of possession. The initial cup is collected by walking into the cup's collision zone, which automatically sets its rendering visibility to hidden and attaches the cup static mesh to a static mesh component attached to the bottom right of the first-person camera called *Item*. This makes the cup permanently visible to the player to reinforce a sense of ownership as they progress through the game.

The second stage of this puzzle only triggers if the player has picked up the first cup. If the player walks into the collision zone of the second cup without having picked up the first, nothing happens. If the player has the first cup and walks into the room of the second cup, player input is disabled and the camera transitions to the dedicated puzzle camera of the second cup *CupCamera* as described in the first approach in **section 3.2.2**. A voiceover presents the second cup as belonging to *Irving*, a non-playable character used for storytelling. The player is informed they can keep one of the two cups and that it is up to them whether they want

to swap. A boolean variable *SecondCupSeen* ensures this cutscene is played just the once, avoiding repetition.

At the end of this cutscene, the camera transitions back to the first-person camera and player input is enabled, leaving the decision in the hands of the player. This setup mirrors the ownership dynamic from the study by Kahneman, Knetsch, and Thaler (1990) by presenting the player with an identical item and asking them to choose between keeping an owned item or swapping for an identical item.

If the player chooses not to swap cups despite both being visually identical, it is interpreted as an indicator of the endowment effect. Their ownership of the initial cup increases its value when compared to an identical alternative.

If the player does choose to swap cups, they can do so by moving closer to the second cup until the *E to interact* widget shows on screen and pressing E while looking at the cup. The illusion of a swap is created by toggling the rendering visibility of both the second cup and the cup in the player's hand with a 0.2 second delay in between, producing a convincing swap mechanic without complex animation. After the cup is swapped, a boolean variable *CupSwapped* is set to true which prevents players from reversing their decision once they have swapped cups. The result is stored in a boolean variable *EndowmentEffect* in the game instance.

3.3.3 Footprints puzzle. The footprints puzzle was designed to measure the player's susceptibility to social proof in a subtle manner. In the study by Milgram, Bickman, and Berkowitz (1969), they demonstrated that observing behavioural cues from others can significantly influence one to mimic the same behaviour [9]. This puzzle attempts to recreate the conditions set in the study by also testing if the presence of more people exhibiting a behavioural cue can influence the actions of oneself more than a single person exhibiting that behaviour.



Figure 6: The footprints puzzle.

This puzzle was designed to subtly test for social proof. In the same cutscene that lets the player know the cup is a gift for them to pick up, the camera is deliberately panned from left to right, providing the player with a clear view of the navigational routes from their position. The left path is visually marked by a single set of footprints, achieved by duplicating a footprint decal actor and flipping the orientation to make it seem like the left side route has been previously traversed by one person. The center path is left

untouched. The right path is marked with several overlapping sets of footprints, achieved in the same way as the left path with the footprint decal used at a higher frequency, creating the impression that the right path is the most traversed route. This setup is subtle in nature to establish an environmental cue without directly instructing the player.

When the player spawns in the level, the current game time in seconds is stored in a float variable *levelStartTime* on the player character. At a certain point on the right-side path, the footprints end abruptly with an indication of the direction the footprints lead to. Continuing in this direction leads the player into a collision box. If the player enters this collision box, the current game time in seconds is retrieved and subtracted with the *levelStartTime* variable to determine the time taken for the player to collide with the box. If the player collides with the box within the first 150 seconds of loading in the level and collides with the intended face of the collision box, the variable *SocialProof* stored in the game instance is set to true. The time constraint ensures that the bias is being displayed as it proves the player follows the footprints early in the playthrough, likely acting based on the environmental cue rather than exploration later in their level progression. The direction constraint is necessary to ensure that only entry from the expected side is considered valid.

The face of the collision box that the player collides with is determined by vector-based calculations. First, the box's forward vector is retrieved and normalized. The box's world position is also stored. The player's world position is then retrieved. By subtracting the box's position from the player's position, a vector from the box to the player is calculated and normalized. Lastly, a dot product between this vector and the box's forward vector is calculated and it determines whether the player collides with the intended face. If the dot product is greater than 0.0, the player enters the box from the intended direction.

In this case, the right-side path's heavy footprint marks serve as the "others" whose behaviour one may mimic. The time and directional constraints make this puzzle a subtle way of testing the player's susceptibility to social proof without any direct player interactions.

3.3.4 Coin flip puzzle. The coin flip puzzle was designed to measure the player's susceptibility to gambler's fallacy. In the study by Tversky and Kahneman (1971), many participants predicted that an overbearing amount of one outcome (e.g, several consecutive heads) made it more likely for the opposite outcome to occur next [10]. This puzzle works directly off that premise and creates the same scenario in game form.



Figure 7: The coin flip puzzle.

A text render on the wall behind the puzzle reads “Flip the coin 5th”. When the player nears the puzzle, the *E to interact* widget appears on the screen. Upon interaction, the camera transitions to the dedicated coin flip camera (CoinFlipCam), *mouseOver* events are enabled, and the cursor is shown on screen. The actor is comprised of three key elements. A coin static mesh, and two buttons labelled ‘heads’ and ‘tails’. They sit on a table constructed from a cube static mesh. To ensure the coin stays on the table, it is surrounded by four invisible walls.

When in CoinFlipCam, clicking on the coin triggers the *FlipCoin* custom event. The event first checks the integer variable *FlipCount*, which tracks the number of times the coin has been flipped. If *FlipCount* is less than 4, a vertical impulse of 375.0 and an angular impulse of 750.0 is added to the coin, animating a convincing coin flip, which is guaranteed to land on heads. This approach was chosen over a timeline-driven Add World Rotation and Add World Location approach to make the coin flip look organic. If *FlipCount* is 4 and the player has not made a prediction, the coin does not flip. Instead a voiceover plays which tells the player to choose a prediction.

At this point the player can now click on the heads button or the tails button to lock in their prediction. Once one of these buttons is pressed, its relative location is lowered and the material of both buttons is greyed out to indicate that the buttons can no longer be pressed. In addition, a boolean operator *ChoiceMade* is set to true. If *ChoiceMade* is true and *FlipCount* is 4, the player can proceed to flip the coin a fifth time. After the fifth flip, any attempts to flip the coin plays a voiceover instructing the player they may move on.

If the player predicts tails for the fifth flip, it is taken as an indicator of the gambler’s fallacy, as the player may believe that tails is “due” despite the independence of coin toss results. The result of the player’s prediction is stored in a *GamblersFallacy* boolean variable in the game instance.

3.3.5 Safe puzzle. The safe puzzle was designed to measure the player’s susceptibility to reverse psychology and reactance. In the study by Brehm (1966), he demonstrated reactance by restricting access to items. He subsequently lifted the restrictions and proved the desire for those restricted items increased [11]. This puzzle attempts to recreate the conditions set in that study.



Figure 8: The safe puzzle.

There is a safe placed on a wall with a text render above that reads “Ethical & Legal violations records”. Next to the safe is a signboard that reads “DO NOT OPEN”, the positioning of this puzzle makes the signboard impossible to miss. This was a deliberate choice to inform the player that access to the safe is restricted. If the player refrains from attempting to open the safe, it is recorded as a sign of resistance to reverse psychology.

An integer variable *safeInteracts*, stored on the player character tracks the number of times the player attempts to open the safe, starting at 0. If the player does try to open the safe, they can approach the safe until the *E to interact* widget appears on screen and press the E Key while looking at the safe. The first time the player does so, the game checks that the value of *safeInteracts* is 0 and plays the *safe sequence*: The first 0.8 seconds of a safe opening sound is played. At 0.8 seconds, the camera view fades to black in 0.1 seconds using *Start Camera Fade*. While the view is black, player input is disabled and the player is placed a few steps away from the safe using *Set World Location*. At this time, a voiceover starts playing as the camera view gradually returns and the player input is enabled after the voiceover ends.

During the first interaction the voiceover explains to the player that the safe is for authorized employees only and asks the player not to try opening the safe again. Simultaneously, *safeInteracts* is incremented to 1. If the player decides to leave at this stage, it is interpreted as an indicator of reverse psychology without any evidence of reactance, as the restriction influences the first attempt but the player refrains from trying again.

If the player tries opening the safe a second time, the game checks the value of *safeInteracts* to be 1, and if it is, it plays the safe sequence again. This time the voiceover is a direct reprimand as “Didn’t we just go over this? Cut it out!” is said to the player. *safeInteracts* is then increased to 2. If the player leaves at this stage, it still reflects susceptibility to reverse psychology but not reactance.

If the player tries opening the safe a third time, the game checks the value of *safeInteracts* to be 2, and if it is, it plays the safe sequence again. This time the voiceover ends with “My boss is calling me... but keep your hands off the safe!”, creating the impression that the

player is no longer being observed and is free to open the safe. `safeInteracts` is then increased to 3. If the player leaves at this stage, the behaviour interpreted remains the same as the previous stages.

The fourth interaction is the crucial one for determining whether the player is susceptible to reactance. For the fourth attempt, the game checks whether `safeInteracts` is 3, and if it is, it plays the safe sequence. While the camera view is black, the safe's rendering visibility and collisions are turned off, as to remove it from the level. The voiceover informs the player that it was a test, and they failed. `safeInteracts` is set to 4 and the puzzle then concludes. Susceptibility to reactance can be drawn from this behaviour as it suggests the player believed the restriction was lifted from the previous voiceover.

The results of this puzzle are stored in two boolean variables, `ReversePsychology` and `Reactance`, stored in the game instance.

3.3.6 Pill puzzle. The pill puzzle was designed to measure the player's susceptibility to hyperbolic discounting and in a second stage, the decoy effect. In the study by Laibson (1997), hyperbolic discounting is described as the tendency to overvalue immediate rewards, leading to inconsistent preferences over time tendency to overvalue immediate rewards, leading to inconsistent preferences over time [14]. Huber, Payne, and Puto's (1982) experiment added a third product that was clearly worse than one of the original two options, and the choice share for that dominant option increased [13]. This puzzle tests both of those premises in the player through their actions.



Figure 9: The pill puzzle.

This puzzle starts with a collision zone, that when collided with, disables player input and triggers a cutscene with the secondary method described in [section 3.2.6](#). The camera transitions to a dedicated puzzle camera (HyperbolicCam), which frames the two pills currently in the scene, pills A and B. Pill C is present in the hierarchy but its rendering and collision are initially off so it doesn't play a role in the first stage of this puzzle. Each pill is placed on its own table marked with a text render to describe the effects of taking the pill. Pill A's table reads "A DOUBLE SPEED NOW", pill B's table reads "B TRIPLE SPEED IN 30 SECONDS".

While the cutscene is playing, the voiceover presents the two pills, stating their effects clearly. The voiceover is interrupted a non-playable character named Dylan who chimes in with "Sir, shouldn't

we tell them that the pills take 50 years off of life expectancy?". This serves to add humour and to incentivize the player to avoid taking a pill, instead opting to try leaving the area. Also, during the cutscene, invisible wall collision boxes are activated at all exits from the area. When the voiceover ends, the camera transitions back to the first-person camera view and player input is enabled. A boolean variable *HyperbolicSeen* ensures this cutscene is only played once.

At this point, the player has two choices. They can either take one pill or attempt to leave without taking any. If they choose to take a pill, it will require them to walk up to the pill of their choosing, waiting for the *E to interact* widget to appear, and press E while looking at their pill of choice. Once the E key is pressed, a function *HandlePillInteract* is called which determines which pill the user takes, updates the *HyperbolicDiscounting* boolean in the game instance, disables rendering for the chosen pill, plays a bite sound indicating the pill has been taken, turns off the exit wall collisions, and sets the *PillTaken* boolean variable to true, preventing the player from taking multiple pills. This function also returns an integer between 1 & 3, which is used to set the movement speed in the first-person character. If at this stage the player chooses pill A, they are displaying susceptibility to hyperbolic discounting.

If the player instead attempts to leave without having taken a pill, when they collide with any of the exit walls, A second cutscene is triggered. The camera transitions back to HyperbolicCam, and the rendering and collision for Pill C is turned on. The voiceover informs the player that Pill C has been added, which triples movement speed in 60 seconds. This is the decoy condition as Pill C is worse than Pill B in every aspect.

Taking a pill calls the same *HandlePillInteract* function from earlier. Choosing Pill A is recorded as the player being susceptible to hyperbolic discounting but not decoy effect, pill B is recorded as the player being susceptible to decoy effect but not hyperbolic discounting and taking pill C is recorded as neither.

3.3.7 Gun puzzle. The gun puzzle was designed to test the player's susceptibility to optimism bias. Sharot (2011) defined optimism bias as "the consistent tendency of people to overestimate the likelihood of experiencing positive events and underestimating the likelihood of experiencing negative ones" [16]. This puzzle attempts to test that theory directly.

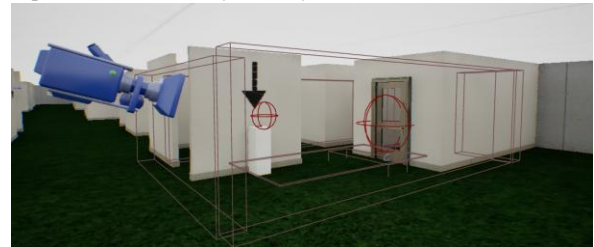


Figure 10: The gun puzzle.

This puzzle was designed to be the first part of a three-part sequence of puzzles. The completion of this puzzle is only

registered when it directly precedes entering the room and having a conversation with Helly.

It starts with a collision zone, that when collided with, disables player input and triggers a cutscene with the secondary method described in **section 3.2.6**. The camera transitions to a dedicated puzzle camera (GunCam), which frames the entire puzzle for the player to see. While the cutscene is playing, the voiceover informs the player “1 in 20 employees, that’s 5%, report that they would have liked some protection... you’re free to go without picking it up... we’re just giving you the option.”. Also, during the cutscene, invisible wall collision boxes are activated at all exits from the area in query only mode. The phrasing of the voiceover is deliberate and emphasizes the voluntary nature of picking up the gun. The choice to proceed without the gun shows an underestimation of risk and is an indicator that the player is susceptible to the optimism bias. A boolean variable *GunSeen* ensures this cutscene only plays once.

When the voiceover ends, the camera transitions back to the first-person camera view and player input is enabled. At this point, the player has the option to pick up the gun or leave the area and come back to it later. To pick up the gun, the player must approach the gun until the *E to interact* widget appears, then look at the gun and press the E key. When the player does this, the rendering visibility of the gun on the table is set to off, a Boolean variable *GunSwapped* is set to true, and a sound plays indicating the player has picked up the gun. If the player tries to leave the area when *GunSwapped* is true and not having entered the room (checked by a boolean variable *TalkedToHelly*, which is set to true after the player has been inside the room and talked to Helly), a voiceover plays informing the player that the gun has been put back where it belongs, at which point the rendering visibility of the gun is set back to on and the player has to pick it up again, if they wish.

Once the player enters the room and collides with the collision box in the room, the game checks whether *GunSwapped* is true and sets *OptimismBias*, a boolean variable in the game instance accordingly. Whether the gun has been picked up or not, the gun’s static mesh visibility on the table is set to off, this is important for the third part in this three-part sequence.

3.3.8 Conversation with Helly. The conversation with Helly was designed to measure the player’s susceptibility to conjunction fallacy. In the study by Tversky and Kahneman (1983), they defined the conjunction fallacy as “the erroneous belief that specific conditions are more probable than a single general one” [18]. This is due to a heavier significance being placed on descriptive representation rather than probabilistic reasoning [18]. This puzzle mirrors the conditions of that study exactly.

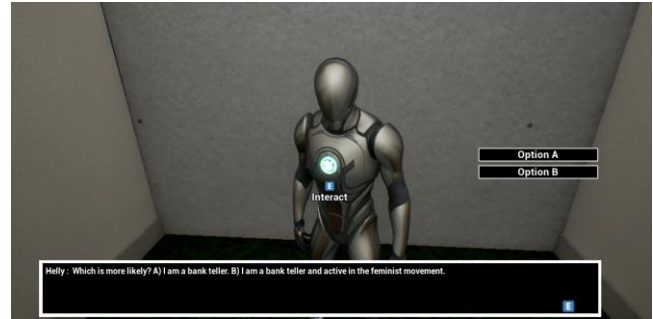


Figure 11: The conversation with Helly.

This puzzle is the second part of the three-part sequence, directly preceding the killing Helly puzzle.

It starts with a collision zone inside the room, that when collided with, locks the door behind the player forcing the player to approach Helly. When approached, an *E to interact* widget appears on screen. When the player presses the E key while looking at Helly, a dialogue appears on screen that introduces Helly as a '31 year old with a financial maths degree and active in issues regarding discrimination, at the bottom right of this dialogue there is an icon of the E key implying that in order to progress, the player must press the E key. Once E is pressed, the dialogue on screen is updated to ask the player whether it is more likely that Helly is a bank teller, or that Helly is a bank teller and active in the feminist movement, along with two buttons 'Option A' and 'Option B'. The mouse cursor is then shown on screen. If the player presses E at this stage, the dialogue, the buttons, and the mouse cursor disappear, but the player only needs to press E again for them to appear again. The player’s answer to this question determines whether they are susceptible to the conjunction fallacy as option A will always be more statistically probable. The button that the player clicks is recorded and saved in a boolean variable *ConjunctionFallacy* in the game instance.

A robust dialogue system has been added in the game using a blueprint interface *Interact*, which has been implemented in the blueprint class *BP_DialogueComponent*. When the player presses E on Helly (whose actor has a *BP_DialogueComponent*), the interact interface is called. A data table *DialogueDataTable* has been created of a custom structure *Dialogue_Structure* type which holds a dialogue, its options (may have none), and whether it is the final dialogue to be shown on screen before destroying the widget. The interact interface begins by checking whether a boolean *EndDialogue* is true, if it isn’t, it then checks whether there is a valid reference to a *WBP_Dialogue* object stored in the *DialogueWidget* variable. Depending on its validity, a widget is either created and added to viewport, or the widget is updated by calling *UpdateDialogue*, *EndDialogue* is updated in both cases for the next press of E when interact is called again.

3.3.9 Killing Helly. The killing Helly puzzle was designed to measure the player’s susceptibility to authority bias. In the study by Milgram (1963), he describes the authority bias as the tendency to

obey orders from an authority figure, even when the order conflicts with one's morals [19]. This puzzle tests the premise of that theory on the player.



Figure 12: The Killing Helly puzzle.

This puzzle is the third and final part of the three-part sequence and occurs directly after the conversation with Helly as the player tries to leave the room.

It starts when the player collides with a collision box a few steps away from Helly, that checks the `TalkedToHelly` Boolean variable set to true immediately after finishing the conversation with Helly puzzle. If it is set to true, player input is disabled and one of two voiceovers play depending on whether the `GunSwapped` variable is true from the gun puzzle. If true, the voiceover dictates "Helly has become a liability to this company, and we need you to eliminate her.". At this point, the red cup static mesh at the bottom right is switched to a gun and a gun-cocking sound is played. The voiceover goes on to explain that the player will face no repercussions if they choose to leave without killing Helly but are being ordered by a direct superior to do so. If `GunSwapped` is false, it plays the same voiceover and does the same sequence but with an added "We snuck the gun in your back pocket while you weren't looking" line added to keep immersion.

At the end of the voiceover, the door to exit the room is unlocked and the boolean variable *KillingHelly* is set to true. While this variable is true, pressing left-click draws a linetrace in the direction of the camera. This is used to check whether the player is aiming at Helly and shooting. At this point, the player can either obey the order even though it may go against their morals thereby proving susceptibility to authority bias, or they may choose to leave the room leaving Helly unharmed.

If they choose to shoot Helly, two functions `HandleHellyKillPtOne` and `HandleHellyKillPtTwo` are called. The former performs the linetrace and checks whether it hits Helly and fades the camera view to black in 0.5 seconds if it does. It returns a reference for Helly. This reference is checked to be valid before it is sent to `HandleHellyKillPtTwo`. This function plays a gunshot sound, destroys the Helly actor, resets the item attached to the camera to a red cup, and sets the boolean variable *AuthorityBias* to true stored in the game instance before it reverses the camera fade. Once the player leaves the room after killing Helly, the door slams shut and locks behind them with an audio cue. This is followed by a voiceover that reaffirms the employee.

If the player chooses not to kill Helly and exits the room, the door slams shut and locks behind them with an audio cue, the static mesh attached to the camera is reset back to a red cup and a voiceover is played which is ominous in nature.

3.3.10 Bottle puzzle. The bottle puzzle was designed to measure the player's susceptibility to the halo effect. In the study by Nisbett and Wilson (1977), they demonstrated the halo effect across multiple domains showing that attractiveness, perceived expertise, or other unrelated qualities could influence judgment in unrelated areas [21].

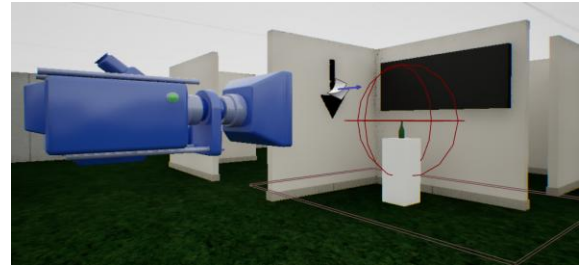


Figure 13: The bottle puzzle.

This puzzle starts with a collision zone, that when collided into, disables player input and triggers a cutscene with the secondary method described in **section 3.2.6**. The camera transitions to a dedicated puzzle camera (BottleCam), which frames the entire puzzle for the player to see. While the cutscene is playing, the voiceover dictates "That's not meant to be part of the puzzle" and then asks 'A-List celebrities' whether they would drink from this bottle. At the end of the voiceover, a video is played on the black cube behind the bottle by a media player component. The video features multiple recognizable celebrities saying the phrase "No way.". Once the video ends, another voiceover informs the player "It is still up to you whether you want to I guess...", leaving the onus on the player. This cutscene only plays once, ensured by a boolean variable *BottleSeen* which is set to true after the cutscene ends. Also at the end of the cutscene, the camera transitions back to first-person view and player input is enabled.

At this point, the player can choose to either drink from the bottle or walk away from the bottle without drinking it. Walking away is interpreted as a behavioural indicator of the halo effect since the player follows the word of the celebrities. If the player chooses to drink from the bottle, they can approach the bottle till the *E to interact* widget appears and press the E key while looking at the bottle.

If they do so, player input is disabled. The bottle's visibility is turned off and after a 0.3 second delay, the cup attached to the camera's visibility is turned off. The static mesh on the table is then swapped to a red cup creating an illusion that the player swapped items. The player's mesh component is then set to visible to display arm animations, and a bottle actor is spawned, attached to the *DrinkSocket* on the left hand. A drinking animation then plays, accompanied by a drinking sound. Once the animation concludes, the spawned bottle actor is destroyed, the cup on the table is hidden, and the cup attached to the camera is made visible again. The static

mesh on the table is switched back to a bottle and made visible. The player mesh component is then hidden and its animation mode is set back to default. To ensure the player can only drink from the bottle once, a boolean variable *HaloEffect* is set in the game instance.

3.4 Explorable areas



Figure 14: Explorable areas exist around the map, the employee pantry is pictured here.

The game features a few explorable areas, irrelevant to the objective of measuring susceptibility to cognitive bias. Their primary purpose is to reinforce the tonal atmosphere of a soulless, comically evil corporate environment presented in a humorous manner. These areas also play a role in improving the free-roaming experience. Without them, the map would feel too sparse. By spreading these areas sporadically across the map, players are more likely to come across at least one.

Each area was crafted using low-poly assets downloaded from marketplaces such as *sketchfab* and *polyhaven*, put together to tell a self-contained story. All areas follow the same method of interaction, each of them have a collision zone that when collided with by the player, plays a snarky voiceover, further reinforcing the tonal atmosphere.

3.5 Menus



Figure 15: The main menu.

The game starts by loading into the level *MainMenu*. *MainMenu*'s event *BeginPlay* is wired to fire a create *WBP_MainMenu* widget and add it to viewport. It consists of a solid black background, a text block displaying the title of the game, and four buttons: New Game, Credits, Profile, and Quit. Selecting New Game loads the *BackroomsLevel0* level, which is the gameplay level. Selecting Credits removes the main menu widget, spawns the credits widget, and adds it to viewport. The credits widget consists of the same black background as the main menu, a text block for credit listings, and a 'Back' button. When the button is clicked, the credits widget is removed and the main menu widget is instantiated again. Selecting Quit closes the application.

An in-game pause menu is also implemented which can be triggered upon pressing the P key. Initially, it displays two options, resume and quit. Resume unpauses the game while selecting Quit expands the menu to reveal two more buttons, Main Menu and Desktop. Main Menu loads the *MainMenu* level and Desktop closes the game directly.

3.6 Profile Screen

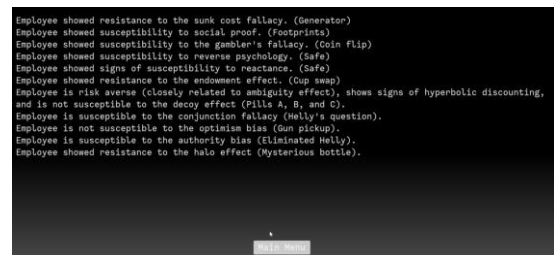


Figure 16: The profile screen.

The profile screen, accessible from both the main menu and at the end of the game, is the screen where the player's cognitive bias susceptibilities are displayed along with the corresponding puzzle completed to measure that susceptibility.

It is identical to the credits widget, except the text is binded to a function which determines the text shown to the player when this widget is created. If the player has not yet completed a full playthrough, a boolean variable *GamePlayed* in the game instance stays false, through which the text displayed to the player is selected to be "Play the game to create a profile."

The function uses a format text node with 13 lines as input. These 13 lines are all hooked up to select text nodes, with the 'Pick A' condition of those nodes being connected to all the cognitive bias boolean variables stored in the game instance. This ensures persistence across levels and that any changes to these biases in subsequent playthroughs are reflected. This format text node with 13 lines is then hooked up to another select text node with the 'Pick A' condition being the aforementioned *GamePlayed* boolean variable, which is only set to true after the player completes a full through and exits through the doorway.

4 Results and evaluation

4.1 Project Objectives

Circling back to the project objectives set at the start of the project.

1. **To design puzzles** that test a player's susceptibility to specific cognitive biases through diegetic game interactions.
2. **To construct an immersive experience** where psychological profiling occurs naturally, without the player feeling any pressure to make a certain decision over another.
3. **To explore diverse game development techniques** in Unreal Engine 5 using blueprints.
4. **To create a humorous but consistent atmosphere** that enhances immersion while showcasing psychological concepts in an engaging way.

Objective 1: *May* have achieved. I can't speak about the objectivity or the scientific relevance of the puzzles designed. The goal was to keep them grounded in the biases tested, and I believe that to be achieved.

Objective 2: *May* have achieved. Every effort has been made to keep the player immersed. However, without having done any player testing, it is unknown whether the world is an immersive experience. It is also unclear whether the average player would feel level-minded during gameplay.

Objective 3: *Achieved*. Having set out to explore as many mechanics and features in UE5 as possible, I have a considerably greater understanding of the engine as a whole and felt my development get better over time.

Objective 4: *May* have achieved. My writing leaves a lot to be desired, the range of comedy feels too narrow, almost repetitive. The atmosphere created is a consistent soulless corporate one, but without player testing I can't speak about how it feels to the average player.

4.2 Performance evaluation



Figure 16: The game running with performance overlays in the unreal PIE.

Performance testing was conducted across three consecutive playthroughs in the Unreal PIE, visually monitoring *stat fps*, *stat unit*, and *stat unitgraph*. Throughout testing, the frame rate

consistently remained above 100 FPS, with frame times never exceeding 13 ms, well within the range of the 60 FPS target. Primitive counts generally remained under 25000, except for when the player got nearer to Helly, when the primitive counts jumped drastically to 50000 when at the gun puzzle, and 285000 during the conversation with Helly. This did not affect the performance noticeably. Memory usage increased from 2.19 GB during the first playthrough to 2.36 GB during the third one. A 170 MB increase indicating asset caching rather than memory leaks. The highest perceptible spike in draw time occurred when loading levels and when going back to the main menu from the profile screen, in both cases it spiked to just above its 70% budget threshold.

Although testing was performed on a relatively-high spec system, the observed performance margins indicate the game is likely to remain stable on mid-range and some lower-end systems with appropriate scalability settings applied.

A low-poly skeletal mesh should have been used for Helly, as the skeletal mesh does not contribute to the game whatsoever, acting as a stand in for two puzzles.

4.3 Known issues

4.3.1 Areas of interest too close. The game assumes the player will slow down, take their time, and inspect all explorable areas. If the player instead decides to have a glimpse and move on, it is possible that the voiceover of an explorable area is still playing when the player comes across a puzzle/another explorable area, playing two voiceovers at the same time. This problem is made especially worse after the pill puzzle if the player's speed is increased.

4.3.2 Talked to Helly boolean. The boolean variable *TalkedToHelly* is set to true when the player selects an option in the conversation with Helly. Selecting an option triggers the section of the blueprint which removes the dialogue widget from the viewport, before immediately setting *TalkedToHelly* to true. There is also an interface *LeaveDialogue* which is intended to keep the player close to Helly while the dialogue is on screen, removing the dialogue if the player wanders too far. I wired this up to the same section of the blueprint as previously mentioned without adding a flag to ensure *TalkedToHelly* isn't set to true in this case. This obvious oversight means it is possible to advance to the Killing Helly puzzle before completing the conversation with Helly.

4.3.3 Issues with input mode UI only. Initially, when creating the pause menu and main menu, I was setting input mode to UI only before setting it back to game mode only when the game was unpaused/in-game. This was causing issues with the mouseOver events in the generator puzzle and coin flip puzzle, requiring a double-click to rotate the crank and flip the coin. After experimenting with all combinations of set input mode, I decided to leave input mode in game mode alone, which means to gain control in-game to look around instead of having the cursor on screen, a single click is required.

5 Conclusions

The aim was to create a game stylistically and thematically similar to *The Stanley Parable*, while incorporating psychological concepts. Although my writing leaves a lot to be desired, I believe I delivered an immersive world with a consistent theme being told through audio and visuals. While the game isn't much to look at, I believe that an engaging style with an interesting story makes a game more enjoyable than visual fidelity. I feel that a walking simulator makes for the ideal theme for a game made to measure susceptibility to cognitive bias, as it is easy to digest and has minimal stimuli, keeping the player in an objective, observant state of mind. The puzzles designed were grounded in their respective biases and each puzzle experimented with something slightly different, to be a training opportunity for myself and a unique experience for the player.

As a learning experience in Unreal Engine, the project went smoothly, I am considerably more knowledgeable in the engine now than I was at the beginning of this project. Certain decisions I made at the start came back to be a pain later. Namely the E to interact system. A good concept, executed poorly by doing it all in the first-person character blueprint. By the end it made it difficult to understand the blueprint workflow. Other decisions I made at the beginning turned out to be great. I made the generator myself by combining multiple static meshes because at the time I did not know how to download and use assets from marketplaces. This paid off because it taught me how to model quickly and proved useful for later development.

I covered the key features in the game in **section 3.2** but my personal favourite mechanic is the dialogue system, it is showcased sparsely but is designed to be infinitely scalable. Some other favourite features of mine include the video playing system, which I had fun experimenting with. Another key feature is the game instance, required for the profile screen to be accessible from both the main menu and the end of the game. It was also important to me to create a main menu, a pause screen and end screen such that the project had the polishing touch of a finished game.

The project was completed to specification with all targeted biases implemented in engaging puzzles and the overall game loop operating smoothly.

5.1 Future work

In its current form, the project functions as proof of concept, demonstrating that cognitive biases can be measured in an entertaining yet minimalistic manner. Future work could expand on this project in several ways.

1. Increasing the number of biases implemented to target more areas of the brain.
2. Dynamically adjusting the world to player behaviour to deepen immersion.
3. VR Implementation to deepen immersion.
4. Multiplayer could accurately recreate conditions set in certain studies exactly.
5. Connecting the game's results to an external database to store and observe the average human being's most common biases.

Polish in visual fidelity, better writing, better sound design, more accessibility options would all improve on this concept and extending the scope in these ways could evolve this project from an academic prototype to a psychological research platform available to the public.

ACKNOWLEDGMENTS

REFERENCES

- [1] Galactic Cafe. 2013. *The Stanley Parable* [PC Game]. Galactic Cafe, United Kingdom.
- [2] Daniel Kahneman. 2011. *Thinking, Fast and Slow* (1st ed.). Farrar, Straus and Giroux, New York, NY.
- [3] UnrealBites. 2022. *Unreal Engine 5: Create a Playable Backrooms Level 0*. YouTube. Retrieved August 8, 2025 from <https://youtu.be/M3VRWBxYrws>.
- [4] John Manoogian III. 2016. *Cognitive Bias Codex* [Infographic]. Wikimedia Commons. Retrieved August 8, 2025 from https://upload.wikimedia.org/wikipedia/commons/6/65/Cognitive_bias_codex_en.svg.
- [5] Hal R. Arkes and Catherine Blumer. 1985. The psychology of sunk cost. *Organizational Behavior and Human Decision Processes* 35, 1 (1985), 124-140. DOI: [https://doi.org/10.1016/0749-5978\(85\)90049-4](https://doi.org/10.1016/0749-5978(85)90049-4).
- [6] Richard H. Thaler. 1980. Toward a positive theory of consumer choice. *Journal of Economic Behavior & Organization* 1, 1 (1980), 39-60. DOI: [https://doi.org/10.1016/0167-2681\(80\)90051-7](https://doi.org/10.1016/0167-2681(80)90051-7).
- [7] Daniel Kahneman, Jack L. Knetsch, and Richard H. Thaler. 1990. Experimental tests of the endowment effect and the Coase theorem. *Journal of Political Economy* 98, 6 (1990), 1325-1348. DOI: <https://doi.org/10.1086/261737>.
- [8] Robert B. Cialdini. 2007. *Influence: The Psychology of Persuasion* (Rev. ed.). Harper Business, New York, NY.
- [9] Stanley Milgram, Leonard Bickman, and Lawrence Berkowitz. 1969. Note on the drawing power of crowds of different size. *Journal of Personality and Social Psychology* 13, 2 (1969), 79-82. DOI: <https://doi.org/10.1037/h0028070>.
- [10] Amos Tversky and Daniel Kahneman. 1971. Belief in the law of small numbers. *Psychological Bulletin* 76, 2 (1971), 105-110. DOI: <https://doi.org/10.1037/h0031322>.
- [11] Jack W. Brehm. 1966. *A Theory of Psychological Reactance*. Academic Press, New York, NY.
- [12] Sharon S. Brehm and Jack W. Brehm. 1981. *Psychological Reactance: A Theory of Freedom and Control*. Academic Press, New York, NY.
- [13] Joel Huber, John W. Payne, and Christopher Puto. 1982. Adding asymmetrically dominated alternatives: Violations of regularity and the similarity hypothesis. *Journal of Consumer Research* 9, 1 (1982), 90-98. DOI: <https://doi.org/10.1086/208899>.
- [14] David Laibson. 1997. Golden Eggs and Hyperbolic Discounting. *Quarterly Journal of Economics* 112, 2 (1997), 443-478. DOI: <https://doi.org/10.1162/003355397555253>.
- [15] Kris N. Kirby and Nino. N. Maraković. 1996. Delay-discounting probabilistic rewards: Rates decrease as amounts increase. *Psychonomic Bulletin & Review* 3, 1 (1996), 100-104. DOI: <https://doi.org/10.3758/BF03210748>.
- [16] Tali Sharot. 2011. The optimism bias. *Current Biology* 21, 23 (2011), R941-R945. DOI: <https://doi.org/10.1016/j.cub.2011.10.030>.
- [17] Neil D. Weinstein. 1980. Unrealistic optimism about future life events. *Journal of Personality and Social Psychology* 39, 5 (1980), 806-820. DOI: <https://doi.org/10.1037/0022-3514.39.5.806>.
- [18] Amos Tversky and Daniel Kahneman. 1983. Extensional versus intuitive reasoning: The conjunction fallacy in probability judgment. *Psychological Review* 90, 4 (1983), 293-315. DOI: <https://doi.org/10.1037/0033-295X.90.4.293>.
- [19] Stanley Milgram. 1963. Behavioral study of obedience. *Journal of Abnormal and Social Psychology* 67, 4 (1963), 371-378. DOI: <https://doi.org/10.1037/h0040525>.

- [20] Edward L. Thorndike. 1920. A constant error in psychological ratings. *Journal of Applied Psychology* 4, 1 (1920), 25-29. DOI: <https://doi.org/10.1037/h0071663>.
- [21] Richard E. Nisbett and Timothy DeCamp Wilson. 1977. The halo effect: Evidence for unconscious alteration of judgments. *Journal of Personality and Social Psychology* 35, 4 (1977), 250-256. DOI: <https://doi.org/10.1037/0022-3514.35.4.250>.